



TECNOLÓGICO
NACIONAL DE MÉXICO



TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE VERACRUZ
INGENIERÍA EN SISTEMAS COMPUTACIONALES

Manual de Sistema

Ofelia Gutiérrez Giraldi

6J1A - Lenguajes y Autómatas I

Equipo 6:

Quintana Pacheco Ángel - 23020808

Sánchez Herrera César Antonio - 23021018

Vargas Valle Marco Antonio - 23020753

ÍNDICE

INTRODUCCIÓN	1
OBJETIVO	3
Objetivo general.....	3
Objetivos específicos.....	3
REQUERIMIENTOS DEL SISTEMA	4
Requerimientos mínimos	4
Requerimientos recomendados	5
INSTALACIÓN Y CONFIGURACIÓN DE JAVACC	6
1. Descarga	6
2. Instalación.....	6
3. Configuración de variables de entorno.....	7
INSTALACIÓN Y CONFIGURACIÓN DE JDK.....	8
1. Descarga	8
2. Instalación.....	8
3. Configuración de variables de entorno.....	8
CONFIGURACIÓN DEL ENTORNO DE TRABAJO	10
COMPILACIÓN Y EJECUCIÓN PASO A PASO	13
TABLA DE TOKENS	15
Flujo del Programa.....	15
Tipo de datos.....	15
Booleanas	16
Condicionales.....	16
Ciclos	17
Arreglos y Matrices.....	17
Operadores de Incremento y decremento.....	17
Operadores aritméticos	18
Operadores adicionales.....	18
Operadores de comparación	18

Comentarios.....	19
Operadores lógicos.....	20
Entrada y salida de datos.....	20
Delimitadores.....	20
Identificadores de variables.....	21
Números Enteros y Decimales.....	22
Cadenas y caracteres.....	22
Tokens de error.....	23
EXPRESIONES REGULARES.....	24
Numéricos.....	24
Texto.....	24
Identificadores.....	25
TOKENS DE ERROR.....	26
Token: ERROR_TEXTO.....	26
Token: ERROR_CARACTER.....	26
Token: ERROR_NUMERO_INVALIDO.....	26
Token: ERROR_IDENTIFICADOR_INVALIDO.....	26
LITERALES.....	27
Estructura del Programa.....	27
Tipos de Datos Y Estructuras de Datos.....	27
Estructuras de Datos.....	27
Valores Booleanos.....	27
Estructuras Condicionales.....	27
Estructuras de Ciclo.....	28
Entrada y Salida.....	28
Operadores de Incremento y Decremento.....	28
Operadores Aritméticos.....	28
Operadores Adicionales.....	28
Operadores de Comparación y Asignación.....	28

Operadores Lógicos	29
Delimitadores	29
GRÁMATICAS	30
Sentencias	30
Bloque anónimo	30
Programa principal	31
Tipo de dato	32
Valores	33
Expresion aritmetica	35
Declaracion de variable	37
Declaración de arreglo	38
Declaración de matriz	39
Asignación	40
Regla para el contenido dentro de IMPRIMIR ()	41
Imprimir	41
Leer	42
Operador relacional	42
Condición simple	43
Condición compuesta	43
Condicional SI CONTRARIO SINO	44
Condicional SEGÚN	45
Bucle PARA	46
Bucle MIENTRAS	47
MANEJO DE ERRORES	48
Creación de lista para almacenar errores	48
Método para obtener posición de error	48
Recuperación de pánico con Sync Tokens	49
Extracción del token que fallo y cual se esperaba	50
Errores Léxicos	51

Error de texto.....	51
Error de caracter.....	51
Error de número invalido.....	51
Error de identificador invalido	52
Error de longitud en caracter.....	52
Error de comparador lógico incompleto.....	52
Errores Sintácticos	53
Error flujo de programa.....	53
Erro declaración de variable.....	53
Error declaración de arreglo.....	54
Error declaración de matriz.....	54
Error de asignación.....	54
Error de impresión.....	55
Error de lectura	55
Error condicional SI.....	55
Error condicional SEGUN.....	56
Error bucle PARA	56
Error bucle MIENTRAS.....	56
CONCLUSIÓN.....	57
REFERENCIAS BIBLIOGRÁFICAS	58

ÍNDICE DE IMÁGENES

IMG 1 UBICACIÓN CARPETA OS(C:).	10
IMG 2 UBICACIÓN CARPETA “USUARIOS”.	10
IMG 3 IDENTIFICAR CARPETA CON NOMBRE DE USUARIO.	11
IMG 4 CREACIÓN DE LA CARPETA.	11
IMG 5 ARCHIVO JJ EN LA CARPETA “EASYCOMPILER”.	12
IMG 6 ENTRAR A CONSOLA	13
IMG 7 ENTRAR A CARPETA DEL COMPILADOR	13
IMG 8 ACTIVAR ACENTOS EN CONSOLA	14
IMG 9 PARSER CON JAVACC	14
IMG 10 COMPILACIÓN DEL CÓDIGO CON JAVA	14
IMG 11 REALIZAR PRUEBA	14
IMG 12 GRAMÁTICA DE SENTENCIAS	30
IMG 13 GRAMÁTICA PARA RECUPERACIÓN DE ERRORES	30
IMG 14 GRAMÁTICA DE PROGRAMA PRINCIPAL	31
IMG 15 GRAMÁTICA PARA TIPOS DE DATOS	32
IMG 16 GRAMÁTICA DE VALORES	33
IMG 17 GRAMÁTICA DE VALORES, POSIBLES ERRORES	34
IMG 18 GRAMÁTICA DE EXPRESIÓN ARITMÉTICA	35
IMG 19 GRAMÁTICA DE EXPRESIÓN ARITMÉTICA	36
IMG 20 GRAMÁTICA PARA DECLARACIÓN DE VARIABLE	37
IMG 21 GRAMÁTICA PARA DECLARACIÓN DE VARIABLE, POSIBLE ERROR	37
IMG 22 GRAMÁTICA PARA DECLARACIÓN DE ARREGLO	38
IMG 23 GRAMÁTICA PARA DECLARACIÓN DE MATRIZ	39
IMG 24 GRAMÁTICA PARA ASIGNACIÓN	40
IMG 25 REGLA PARA IMPRIMIR	41
IMG 26 GRAMÁTICA PARA IMPRIMIR	41
IMG 27 GRAMATICA PARA LEER	42
IMG 28 GRAMÁTICA PARA OPERADOR RELACIONAL	42
IMG 29 GRAMÁTICA PARA CONDICIÓN SIMPLE	43
IMG 30 GRAMÁTICA PARA CONDICIÓN COMPUESTA	43
IMG 31 GRAMÁTICA PARA CONDICIONAL SI CONTRARIO SINO	44
IMG 32 GRAMÁTICA PARA CONDICIONAL SEGÚN	45

IMG 33 GRAMÁTICA PARA BUCLE PARA	46
IMG 34 GRAMÁTICA PARA BUCLE MIENTRAS	47
IMG 35 LISTA PARA MANEJO DE ERRORES	48
IMG 36 MÉTODO PARA UBICAR ERROR	48
IMG 37 MÉTODO PARA RECUPERACIÓN DE PÁNICO	49
IMG 38 EXTRACCIÓN DE TOKEN ERRONEO	50
IMG 39 ERROR DE TEXTO	51
IMG 40 ERROR DE CHARACTER	51
IMG 41 ERROR DE NÚMERO INVALIDO	51
IMG 42 ERROR DE IDENTIFICADOR INVALIDO	52
IMG 43 ERROR DE LONGITUD EN CHARACTER	52
IMG 44 ERROR DE COMPARADOR LÓGICO INCOMPLETO	52
IMG 45 ERROR DE FLUJO DE PROGRAMA	53
IMG 46 ERROR DE DECLARACIÓN DE VARIABLES	53
IMG 47 ERROR DE DECLARACIÓN DE ARREGLO	54
IMG 48 ERROR DE DECLARACIÓN DE MATRIZ	54
IMG 49 ERROR DE ASIGNACIÓN	54
IMG 50 ERROR DE IMPRESIÓN	55
IMG 51 ERROR DE LECTURA	55
IMG 52 ERROR DE CONDICIONAL SI	55
IMG 53 ERROR DE CONDICIONAL SEGUN	56
IMG 54 ERROR DE BUCLE PARA	56
IMG 55 ERROR DE BUCLE MIENTRAS	56

INTRODUCCIÓN

EasyCompiler es una herramienta educativa que integra un lenguaje de programación diseñado para el aprendizaje y su correspondiente compilador, desarrollada como parte de un proyecto académico con el propósito de facilitar la enseñanza de los fundamentos de la programación a estudiantes hispanohablantes.

El nombre EasyCompiler surge de la combinación de la palabra inglesa "Easy" (fácil) y "Compiler" (compilador), formando así un término que refleja la filosofía central del proyecto: proporcionar un entorno de programación sencillo y accesible para quienes se inician en el mundo de la programación.

La elección de este nombre simboliza el compromiso de ofrecer una herramienta que elimine barreras técnicas e idiomáticas, permitiendo que los estudiantes puedan concentrarse en comprender la lógica de la programación sin las complejidades adicionales que suelen presentar los lenguajes y entornos profesionales.

El presente manual de sistema está dirigido a todas aquellas personas que utilizarán EasyCompiler como herramienta de aprendizaje o enseñanza de la programación. En él se describen las características del lenguaje, la estructura de sus programas, las instrucciones para la instalación y uso del compilador, así como la interpretación de los mensajes que este genera. Este documento busca ser una guía completa que acompañe al estudiante en su proceso formativo y al docente en su labor pedagógica.

EasyCompiler se distingue por su enfoque en la claridad y la accesibilidad, convirtiéndose en un recurso valioso para la enseñanza introductoria de la programación. El lenguaje incorpora una sintaxis intuitiva con palabras reservadas en español, lo que reduce la barrera idiomática y permite a los estudiantes hispanohablantes comprender más fácilmente la estructura y lógica de un programa.

El compilador, por su parte, cuenta con un sistema de gestión de errores que genera mensajes descriptivos en español, guiando al estudiante en la detección y corrección de sus propios errores. Este enfoque no solo facilita la identificación de fallos, sino que promueve el aprendizaje autónomo y una comprensión más profunda de las reglas del lenguaje,

fomentando la experimentación constante y el desarrollo de habilidades fundamentales de programación en un entorno controlado y didáctico.

OBJETIVO

Objetivo general

Proporcionar a estudiantes y docentes una herramienta educativa integral, compuesta por un lenguaje de programación en español y su compilador asociado, que facilite el aprendizaje y la enseñanza de los fundamentos de la programación mediante un entorno sencillo, accesible y con mensajes de error claros que promuevan la autonomía del estudiante.

Objetivos específicos

1. Presentar la sintaxis y estructura del lenguaje de programación EasyCompiler, incluyendo su conjunto de palabras reservadas en español, tipos de datos, operadores y estructuras de control, para que el usuario pueda comprender y utilizar correctamente el lenguaje.
2. Explicar el proceso de instalación, configuración y uso del compilador EasyCompiler, detallando los comandos disponibles y las opciones de compilación para que el usuario pueda poner en funcionamiento la herramienta sin dificultades.
3. Describir el sistema de mensajes de error del compilador, proporcionando una guía de interpretación y corrección que permita al estudiante identificar y resolver de manera autónoma los errores en sus programas.
4. Ilustrar mediante ejemplos prácticos y casos de uso la aplicación de los conceptos de programación en el lenguaje EasyCompiler, facilitando la comprensión de temas fundamentales como variables, condicionales, ciclos y funciones.
5. Servir como material de referencia y apoyo para cursos introductorios de programación, ofreciendo a docentes y estudiantes un recurso completo que describa el funcionamiento y las capacidades de las herramientas EasyCompiler.

REQUERIMIENTOS DEL SISTEMA

Para garantizar el correcto funcionamiento de EasyCompiler, es necesario que el equipo donde se ejecute cumpla con los siguientes requisitos:

Requerimientos mínimos

Los requerimientos mínimos permiten ejecutar el compilador de manera funcional, aunque el rendimiento puede verse afectado al compilar programas más complejos o de mayor tamaño.

Componente	Especificación
Procesador	Intel Core i3 o equivalente (1.5 GHz)
Memoria RAM	2 GB
Almacenamiento	50 MB de espacio libre en disco
Sistema Operativo	Windows 7/8/10/11
Java	Java Development Kit (JDK) 8 o superior
JavaCC	JavaCC 7.0.13
Terminal	Acceso a línea de comandos (CMD)
Editor de texto	Bloc de notas
Resolución de pantalla	1024 x 768 píxeles

Requerimientos recomendados

Los requerimientos recomendados aseguran una experiencia óptima, especialmente al trabajar con programas extensos o al utilizar el compilador de forma intensiva en entornos educativos.

Componente	Especificación
Procesador	Intel Core i5 o superior (2.0 GHz o más)
Memoria RAM	4 GB o más
Almacenamiento	100 MB de espacio libre en disco
Sistema Operativo	Windows 10/11
Java	Java Development Kit (JDK) 17 o superior
Javacc	JavaCC 7.0.13
Terminal	cmd.exe
Editor de texto	Notepad++
Resolución de pantalla	1920 x 1080 píxeles o superior

INSTALACIÓN Y CONFIGURACIÓN DE JAVACC

1. Descarga

Paso 1: Accede al sitio oficial de JavaCC: <https://javacc.github.io/javacc/>

Paso 2: Descarga el código fuente en formato ZIP haciendo clic en la opción JavaCC 7.0.13 ZIP (ubicada en el centro de la página).

Paso 3: En esa misma página principal, haz clic en el botón "View on GitHub".

Paso 4: Dentro del repositorio de GitHub, desplázate hacia abajo hasta localizar el archivo README.

Paso 5: En el README, busca el apartado "Version 7" y haz clic en el enlace "Binaries" para descargar el archivo ejecutable .jar.

Paso 6: Abre tu explorador de archivos (usualmente en la carpeta "Descargas") y verifica que tengas ambos archivos: el .zip y el .jar.

2. Instalación

Paso 1: Descomprime el archivo ZIP descargado. Para ello, haz clic derecho sobre él y selecciona "Extraer todo...".

Paso 2: Entra a la carpeta que se acaba de descomprimir (llamada javacc-javacc-7.0.13). Dentro de ella, crea una nueva carpeta llamada **target** (preferentemente todo en minúsculas para evitar errores).

Paso 3: Mueve o copia el archivo .jar (el que descargaste desde GitHub) dentro de esta nueva carpeta target.

Paso 4: Haz clic derecho sobre el archivo .jar que acabas de mover, selecciona "Cambiar nombre" y nómbralo exactamente **javacc.jar**.

Nota*: Si tu explorador de Windows oculta las extensiones de archivo y no ves el ".jar" al final de los nombres, simplemente nómbralo javacc.

3. Configuración de variables de entorno

Paso 1: Ve a la carpeta principal descomprimida (javacc-javacc-7.0.13), entra a la subcarpeta llamada scripts y copia la ruta completa desde la barra de direcciones superior del explorador de archivos.

Paso 2: Abre el buscador de Windows (tecla Windows), escribe “variables de entorno” y selecciona la opción “Editar las variables de entorno del sistema”.

Paso 3: En la ventana que se abre, haz clic en el botón inferior llamado “Variables de entorno...”.

Paso 4: En la sección inferior llamada “Variables del sistema”, busca en la lista la variable llamada Path, selecciónala con un clic y luego presiona el botón “Editar...”.

Paso 5: Haz clic en el botón "Nuevo" y pega la ruta de la carpeta scripts que copiaste en el Paso 1. Luego haz clic en Aceptar en todas las ventanas abiertas para guardar los cambios.

Paso 6: Abre una nueva ventana de la terminal (presiona Win, escribe cmd y dale Enter). Es crucial que sea una ventana nueva para que reconozca los cambios.

Paso 7: Escribe el siguiente comando para verificar que la instalación fue exitosa:

```
javacc --version
```

INSTALACIÓN Y CONFIGURACIÓN DE JDK

1. Descarga

Paso 1: Accede a la página oficial de descargas de Oracle Java:

<https://www.oracle.com/mx/java/technologies/downloads/>

Paso 2: Busca la pestaña “Java SE Development Kit” y elige la versión de soporte a largo plazo (LTS) más reciente. Luego, selecciona el instalador correspondiente a tu sistema operativo.

Para Windows, elige la opción "x64 Installer" (descargará un archivo .exe).

Paso 3: Haz clic en el enlace de descarga, acepta los términos y condiciones de Oracle (si la página lo solicita), y espera a que el archivo se descargue por completo.

2. Instalación

Paso 1: Ve a tu carpeta de descargas, haz clic derecho sobre el archivo instalador del JDK (ej. jdk-21_windows-x64_bin.exe) y selecciona “Ejecutar como administrador”.

Paso 2: En la ventana de bienvenida del instalador, presiona “Next” (Siguiente).

Paso 3: La pantalla mostrará la ruta donde se instalará el JDK (por defecto suele ser C:\Program Files\Java\jdk-xx). Es recomendable dejar esta ruta por defecto. Haz clic en “Next” para comenzar la instalación.

Paso 4: Una vez que la barra de progreso termine y veas el mensaje de instalación exitosa, presiona el botón “Close” para cerrar el instalador.

3. Configuración de variables de entorno

Paso 1: Abre el explorador de archivos y navega hasta la carpeta donde se instaló el JDK, y entra específicamente a la carpeta bin.

La ruta exacta que debes copiar de la barra superior debería verse similar a esta:

C:\Program Files\Java\jdk-21\bin

(Asegúrate de que termine en \bin, ya que ahí viven los ejecutables).

Paso 2: Abre el buscador de Windows (presiona la tecla Windows), escribe “variables de entorno” y selecciona la opción “Editar las variables de entorno del sistema”.

Paso 3: En la ventana de Propiedades del sistema, haz clic en el botón inferior que dice “Variables de entorno...”.

Paso 4: En el panel de abajo llamado “Variables del sistema”, busca en la lista la variable llamada Path. Haz clic sobre ella para seleccionarla y luego presiona el botón “Editar...”.

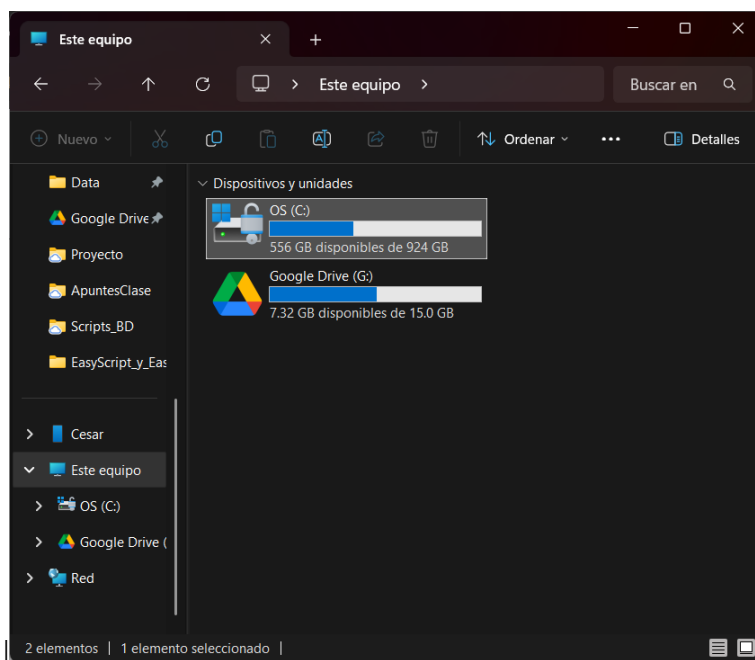
Paso 5: En la nueva ventana, haz clic en el botón "Nuevo". Se habilitará un espacio vacío al final de la lista; pega ahí la ruta de la carpeta bin que copiaste en el Paso 1. Haz clic en "Aceptar" en todas las ventanas que abriste para guardar los cambios.

Paso 6: Abre una nueva ventana de la terminal (escribe cmd en el buscador de Windows y dale Enter). Escribe el siguiente comando para verificar la instalación:

```
java -version
```

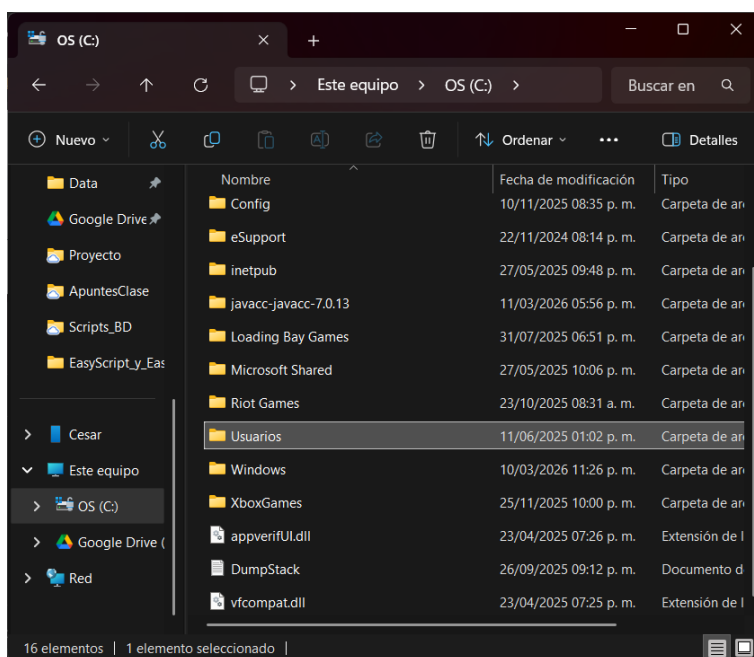

CONFIGURACIÓN DEL ENTORNO DE TRABAJO

1. Abrir el explorador de archivos.
2. Entrar a la carpeta OS(C:) en “Este equipo” y.



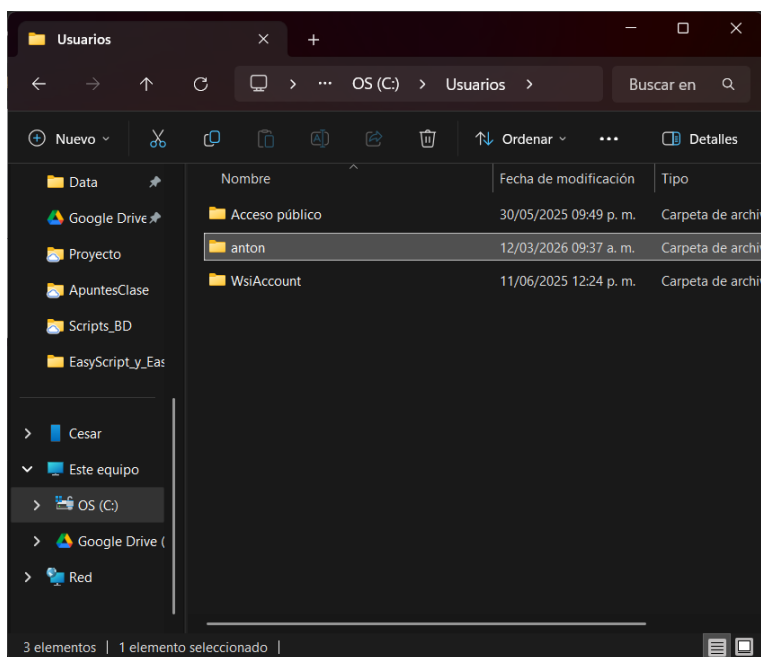
Img 1 Ubicación carpeta OS(C:).

3. Entrar a la carpeta “usuarios”.



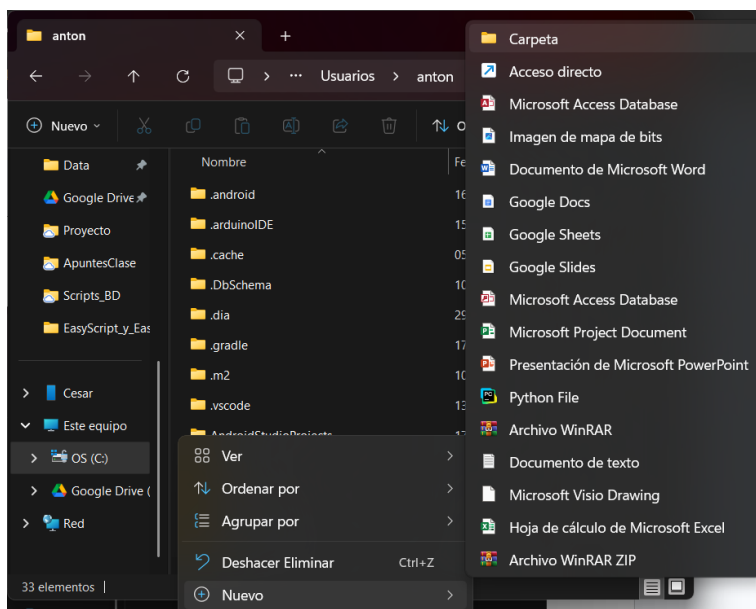
Img 2 Ubicación carpeta “Usuarios”.

4. Identificar la carpeta con el nombre del usuario que está en uso del dispositivo y entrar.



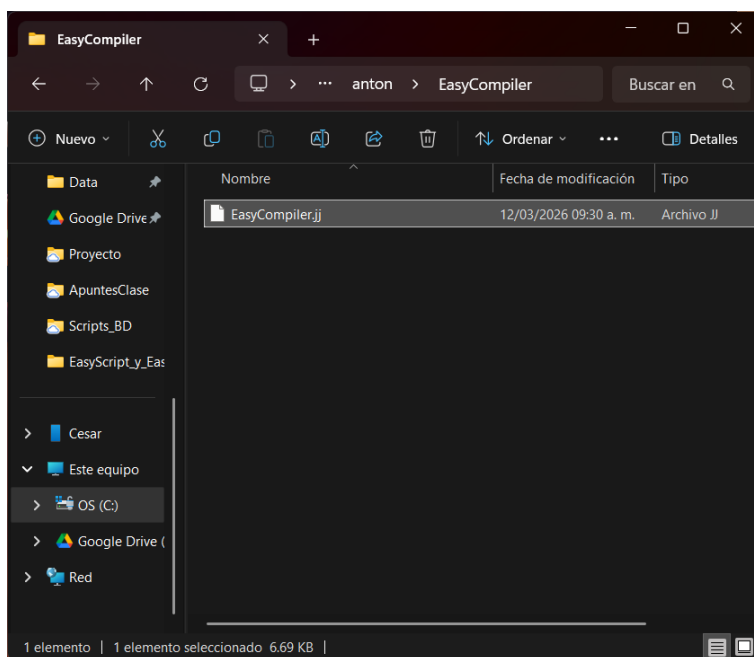
Img 3 Identificar carpeta con nombre de usuario.

5. Crear una carpeta con el nombre del compilador “EasyCompiler”.



Img 4 Creación de la carpeta.

6. Dentro de la carpeta recién creada “EasyCompiler” pegar el archivo “EasyCompiler.jj”.

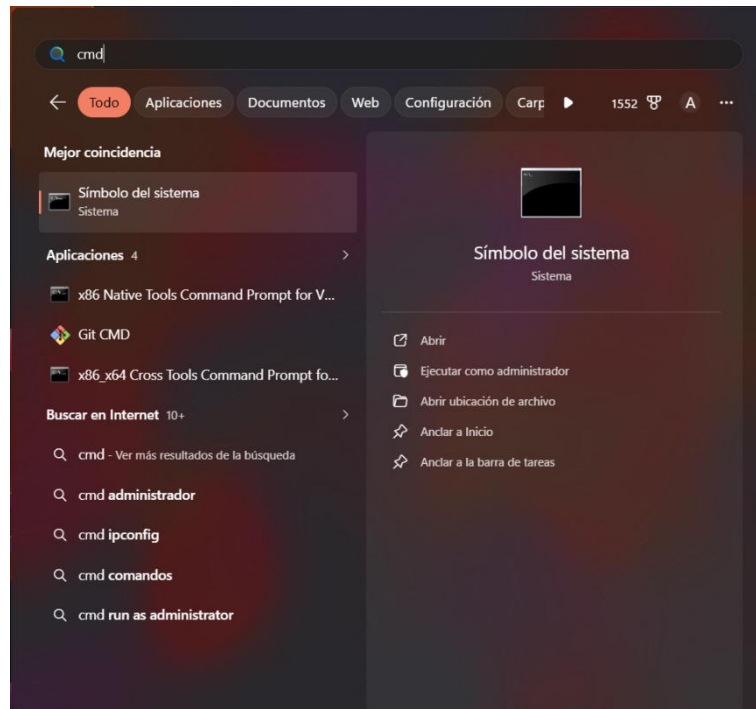


Img 5 Archivo jj en la carpeta “EasyCompiler”.

Nota*: En esta misma carpeta se deben colocar los archivos de prueba.

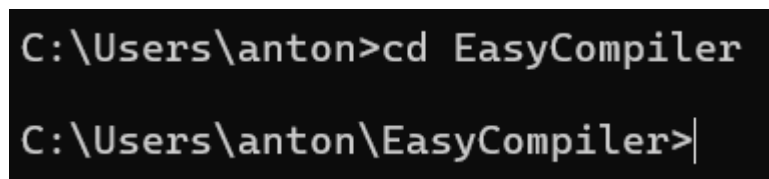
COMPILACIÓN Y EJECUCIÓN PASO A PASO

1. Presiona tecla Windows y buscar CMD para entrar a la consola.



Img 6 Entrar a consola

2. Entrar a la carpeta “EasyCompiler” creada anteriormente.



Img 7 Entrar a carpeta del compilador

3. Activar UFT-8 para ver los acentos correctamente.

```
C:\Users\anton\EasyCompiler>chcp 65001
Página de códigos activa: 65001

C:\Users\anton\EasyCompiler>
```

Img 8 Activar acentos en consola

4. Generar el parser con JavaCC.

```
C:\Users\anton\EasyCompiler>javacc EasyCompiler.jj
Java Compiler Compiler Version 7.0.13 (Parser Generator)
(type "javacc" with no arguments for help)
Reading from file EasyCompiler.jj . . .
File "TokenMgrError.java" is being rebuilt.
File "ParseException.java" is being rebuilt.
File "Token.java" is being rebuilt.
File "SimpleCharStream.java" is being rebuilt.
Parser generated successfully.

C:\Users\anton\EasyCompiler>|
```

Img 9 Parser con JavaCC

5. Compilar el código con Java.

```
C:\Users\anton\EasyCompiler>javac *.java

C:\Users\anton\EasyCompiler>
```

Img 10 Compilación del código con Java

6. Ejecutar un archivo de prueba en el compilador.

```
C:\Users\anton\EasyCompiler>java Main codigo1.txt
```

Img 11 Realizar prueba

TABLA DE TOKENS

Flujo del Programa

SÍMBOLO	TOKEN	DESCRIPCIÓN
INICIO	INICIO_PROGRAMA	Define el punto de inicio y/o entrada del programa. Todo código ejecutable debe estar después de este token y antes de fin. Es obligatorio en cualquier programa.
FIN	FIN_PROGRAMA	Marca el final del programa. Después de esta palabra no puede haber código ejecutable. Es obligatorio en cualquier programa.

Tipo de datos

SÍMBOLO	TOKEN	DESCRIPCIÓN
ENT	ENTERO_DATO	Representa números enteros sin componente fraccionario. Utilizado para contadores y cantidades discretas.
DEC	DECIMAL_DATO	Representa números de punto flotante, aptos para valores que requieren precisión o tienen una componente fraccionaria.
BOOL	BOOLEANO_DATO	Representa valores de verdad, limitado a dos estados: VERDADERO o FALSO . Utilizado en lógica condicional.
LETRA	CARACTER_DATO	Almacena un único carácter (letra, número, símbolo). Se encierra entre comillas simples ('a').
TXT	TEXTO_DATO	Almacena una secuencia de caracteres (cadenas de texto). Se encierra entre comillas dobles ("Hola Mundo").

Booleanas

SÍMBOLO	TOKEN	DESCRIPCIÓN
VERDADERO	VERDADERO_BOOLEAN O	Muestra que la condición es verdadera.
FALSO	FALSO_BOOLEANO	Muestra que la condición es falsa.

Condicionales

SÍMBOLO	TOKEN	DESCRIPCIÓN
SI	SI_CONDICIONAL	Si la condición es verdadera ejecuta un bloque de código asociado.
CONTRARIO	SI_CONTRARIO	Define un bloque anidado dentro del bloque principal del SI.
SINO	SI_SINO	Define un bloque alternativo que se ejecuta cuando la condición SI es falsa.
SEGUN	SEGUN_CONDICIONAL	Condicional que permite tener opciones de casos determinados.
CASO	CASO_SEGUN	Palabra reservada para definir el caso con una variable.
DEFECTO	DEFECTO_SEGUN	Palabra reserva para indicar que la condicional SEGUN tenga una variable por defecto en caso que los demás casos hayan sido

SÍMBOLO	TOKEN	DESCRIPCIÓN
		falsos.
DETENER	DETENER_SEGUN	Palabra reservada para indicar una detención en un caso.

Ciclos

SÍMBOLO	TOKEN	DESCRIPCIÓN
PARA	PARA_CICLO	Ciclo con contador que combina inicialización de variable, condición de continuación y actualización del contador.
MIENTRAS	MIENTRAS_CICLO	Ciclo que repite una sentencia mientras su condición sea falsa.

Arreglos y Matrices

SÍMBOLO	TOKEN	DESCRIPCIÓN
ARREGLO	ARREGLO_ESTRUCTURA	Palabra reservada para la creación de un arreglo.
MATRIZ	MATRIZ_ESTRUCTURA	Palabra reservada para la creación de una matriz.

Operadores de Incremento y decremento

SÍMBOLO	TOKEN	DESCRIPCIÓN
++	INCREMENTO_AL_VALOR	Aumenta el valor de una

SÍMBOLO	TOKEN	DESCRIPCIÓN
		variable en 1.
--	DECREMENTO_AL_VALOR	Disminuye el valor de una variable en 1.

Operadores aritméticos

SÍMBOLO	TOKEN	DESCRIPCIÓN
+	SUMA	Suma valores numéricos.
-	RESTA	Resta valores numéricos.
*	MULTIPLICACION	Multiplica los valores numéricos.
/	DIVISION	Divide números y retorna un resultado entero o decimal.

Operadores adicionales

SÍMBOLO	TOKEN	DESCRIPCIÓN
%	MODULO	Saca el módulo de una división.
**	EXPONENTE	Permite tener el resultado de un exponente elevado a un número dado.

Operadores de comparación

SÍMBOLO	TOKEN	DESCRIPCIÓN
=	IGUAL_ASIGNACION	Asigna el valor de la derecha a la variable de la

SÍMBOLO	TOKEN	DESCRIPCIÓN
		izquierda.
==	IGUAL_COMPARACION	Compara si dos valores son exactamente iguales. No es lo mismo que = (asignación).
!=	DIFERENTE	Verifica si dos valores son diferentes.
<	MENOR	Verifica si el valor izquierdo es estrictamente menor que el derecho.
<=	MENOR_IGUAL	Verifica si el valor izquierdo es estrictamente menor o igual que el derecho.
>	MAYOR	Verifica si el valor izquierdo es estrictamente mayor que el derecho.
>=	MAYOR_IGUAL	Verifica si el valor izquierdo es estrictamente menor o igual que el derecho.

Comentarios

SÍMBOLO	TOKEN	DESCRIPCIÓN
//	COMENTARIO	Ignora todo lo que este después de él para funcionar como comentario.
/*	ABRIR_BLOQUE _COMENTARIO	Inicia un bloque de comentarios, es decir, conjunto de líneas comentadas.
*/	CERRAR_BLOQUE _COMENTARIO	Termina el bloque de comentarios, debe corresponder con el símbolo de apertura.

Operadores lógicos

SÍMBOLO	TOKEN	DESCRIPCIÓN
&&	Y_LOGICO	Operador AND, retorna verdadero solo si ambas condiciones.
 	O_LOGICO	Operador OR, retorna verdadero si al menos una condición es verdadera.
!!	NO_LOGICO	Operador NOT, invierte el valor lógico: verdadero→falso, falso→verdadero.

Entrada y salida de datos

SÍMBOLO	TOKEN	DESCRIPCIÓN
LEER	LEER_FUNCION	Palabra reservada para entrada de datos.
IMPRIMIR	IMPRIMIR_FUNCION	Sirve para mostrar la salida de datos.

Delimitadores

SÍMBOLO	TOKEN	DESCRIPCIÓN
{	ABRIR_LLAVE	Inicia un bloque de código agrupando y definiendo un alcance de estructuras y funciones.

SÍMBOLO	TOKEN	DESCRIPCIÓN
}	CERRAR_LLAVE	Termina un bloque de código, debe corresponder con una llave de apertura
(	ABRIR_PARENTESIS	Inicia una agrupación, usado para condiciones, parámetros y procedencia.
)	CERRAR_PARENTESIS	Cierra agrupación, debe corresponder con un paréntesis de apertura.
[	ABRIR_CORCHETE	Inicia definición de arreglo o acceso a elemento.
]	CERRAR_CORCHETE	Cierra definición de arreglo o acceso a elemento
,	COMA	Separa elementos.
:	DOS_PUNTOS	Marca el inicio para un caso de la condicional SEGUN.
;	PUNTO_COMA	Marca el final de una instrucción.

Identificadores de variables

EXPRESIÓN	TOKEN	DESCRIPCIÓN
["A"- "Z", "Á", "É", "Í", "Ó", " Ú", "Ñ"] (["a"- "z", "0"- "9", " _", "á", "é", "í", "ó", "ú", "ñ"])*	VARIABLE	Identificador de Variable, la variable de de empezar con una Letra mayúscula y no debe de empezar con un número. Puede incluir acentos del español y la letra ñ / Ñ.
"CONST_" ["A"-	CONSTANTE	Identificador de constante. debe de iniciar

EXPRESIÓN	TOKEN	DESCRIPCIÓN
"Z", "Á","É","Í","Ó", "Ú","Ñ" (["a"- "z", "0"- "9", "_", "á","é","í","ó","ú", "ñ"])*		con "CONST_" seguido de la estructura de variable.

Números Enteros y Decimales

EXPRESIÓN	TOKEN	DESCRIPCIÓN
["0"- "9"]+	NUM_ENTERO	Número entero, puede tener combinaciones entre el 0-9.
(["0"- "9"]+ "." (["0"- "9"]+) ("." (["0"- "9"]+)	NUM_DECIMAL	Número decimal, puede tener o no combinaciones entre el 0-9 seguido de un punto decimal y terminando con combinaciones entre el 0-9.

Cadenas y caracteres

EXPRESIÓN	TOKEN	DESCRIPCIÓN
"\" ((~["\"","\\","\n", "r"]) ("\" ["n","t","r","\n", ""]))* "\"	CADENA	Cadena. Todo lo que esté entre comillas dobles. Acepta cualquier carácter normal excepto comillas, barras invertidas o saltos de línea físicos o acepta una barra invertida seguida de n, t, r.
"" ["a"- "z", "A"- "Z", "0"-	VALOR_LETRA	Valor para una letra. Solo acepta una letra en mayúscula o minúscula o un numero

"9"] ""		entre el 0-9 dentro de comillas simples.
---------	--	--

Tokens de error

TIPO	TOKEN	¿DÓNDE?	DESCRIPCIÓN
Léxico	ERROR_TEXTO	Línea y Columna.	Cadena no cerrada con comillas dobles
Léxico	ERROR_CHARACTER	Línea y Columna.	Carácter no cerrado con comillas simples.
Léxico	ERROR_NUMERO_INVALIDO	Línea y Columna.	Formato de número decimal inválido.
Léxico	ERROR_IDENTIFICADOR_INVALIDO	Línea y Columna.	Identificador (Variable o Constante) que comienza con letra en minúscula o número.
Léxico	ERROR_SIMBOLO	Línea y Columna.	Símbolo no válido en el lenguaje.
Léxico	ERROR_LONGITUD_CHARACTER	Línea y Columna	Atrapa textos encerrados en comillas simples que tienen más de una letra (ej. 'Numero') o ninguna (").
Léxico	ERROR_OPERADOR_INCOMPLETO	Línea y Columna	Atrapa símbolos que deberían ir dobles en EasyScript, pero se escribieron solos.

EXPRESIONES REGULARES

Numéricos

- Token: NUM_ENTERO
 - Expresión Regular:

$$(["0" - "9"])+$$
 - Valores permitidos: Número entero que puede tener combinaciones entre los dígitos del 0 al 9.
- Token: NUM_DECIMAL
 - Expresión Regular:

$$((["0" - "9"])+ \cdot ("["0" - "9"])+) | (\cdot ("["0" - "9"])+)$$
 - Valores permitidos: Número decimal que puede tener o no combinaciones entre el 0-9 antes del punto decimal, pero debe terminar con combinaciones entre el 0-9 después del punto.

Texto

- Token: VALOR_LETRA
 - Expresión Regular:

$$"" ["a" - "z", "A" - "Z", "0" - "9"] ""$$
 - Valores permitidos: Un solo carácter (letra en mayúscula o minúscula) o número encerrado entre comillas simples.
- Token: CADENA
 - Expresión Regular:

$$""\ "" ((\sim["\ "" , "\\", "\n", "\r"]) | ("\" ["n", "t", "r", "\\", "\"])) * ""\ ""$$
 - Valores permitidos: Cualquier secuencia de caracteres entre comillas dobles; permite caracteres normales (excepto comillas, barras invertidas o saltos de línea físicos). Secuencias de escape: \n, \t, \r.

Identificadores

- Token: VARIABLE

- Expresión Regular:

$["A" - "Z", "Á", "É", "Í", "Ó", "Ú", "Ñ"] (["a" - "z", "0" - "9", "_", "á", "é", "í", "ó", "ú", "ñ"]) *$

- Valores permitidos: Identificador que debe comenzar obligatoriamente con una letra mayúscula y puede continuar con letras, números o guiones bajos; no puede iniciar con un número.

- Token: CONSTANTE

- Expresión Regular:

$"CONST_" ["A" - "Z", "Á", "É", "Í", "Ó", "Ú", "Ñ"] (["a" - "z", "0" - "9", "_", "á", "é", "í", "ó", "ú", "ñ"]) *$

- Valores permitidos: Debe iniciar con el prefijo CONST_ seguido de la estructura permitida para una variable.

TOKENS DE ERROR

Token: ERROR_TEXTO

- Expresión Regular:

`"\" (~["\"", "\n", "\r"])*`

Abre comillas, tiene texto, pero choca con un salto de línea antes de cerrar comillas.

Token: ERROR_CARACTER

- Expresión Regular:

`'\" (~["\"", "\n", "\r"])*`

Abre comilla simple, tiene contenido, pero choca con salto de línea antes de cerrar.

Token: ERROR_NUMERO_INVALIDO

- Expresión Regular:

`(["0" – "9"]) + "." (["0" – "9"]) + ("." (["0" – "9"])+) +`

Atrapa números con múltiples puntos decimales, ej. 3.14.15.

Token: ERROR_IDENTIFICADOR_INVALIDO

- Expresión Regular:

Atrapa identificadores que comienzan con número. ejemplo 9numero.

`(["0" – "9"] (["a" – "z", "A" – "Z", "0" – "9", "_", "á", "é", "í", "ó", "ú", "Á", "É", "Í", "Ó", "Ú", "ñ", "Ñ"])+ )`

O que comienzan con minúscula. ejemplo: numero.

`| (["a" – "z", "á", "é", "í", "ó", "ú", "ñ"] (["a" – "z", "A" – "Z", "0" – "9", "_", "á", "é", "í", "ó", "ú", "Á", "É", "Í", "Ó", "Ú", "ñ", "Ñ"])* )`

O que tengan más de una letra en mayúscula. ejemplo: PrimerNumero.

`| (["A" – "Z", "Á", "É", "Í", "Ó", "Ú", "Ñ"] (["a" – "z", "0" – "9", "_", "á", "é", "í", "ó", "ú", "ñ"])* ["A" – "Z", "Á", "É", "Í", "Ó", "Ú", "Ñ"] (["a" – "z", "A" – "Z", "0" – "9", "_", "á", "é", "í", "ó", "ú", "Á", "É", "Í", "Ó", "Ú", "ñ", "Ñ"])* )`

LITERALES

Estructura del Programa

- **INICIO_PROGRAMA:** "INICIO"
- **FIN_PROGRAMA:** "FIN"
- **LEER_DATO:** "LEER"
- **IMPRIMIR_DATO:** "IMPRIMIR"

Tipos de Datos Y Estructuras de Datos

- **ENTERO_DATO:** "ENT"
- **DECIMAL_DATO:** "DEC"
- **BOOLEANO_DATO:** "BOOL"
- **CARACTER_DATO:** "LETRA"
- **TEXTO_DATO:** "TXT"

Estructuras de Datos

- **ARREGLO_ESTRUCTURA:** "ARREGLO"
- **MATRIZ_ESTRUCTURA:** "MATRIZ"

Valores Booleanos

- **VERDADERO_BOOLEANO:** "VERDADERO"
- **FALSO_BOOLEANO:** "FALSO"

Estructuras Condicionales

- **SI_CONDICIONAL:** "SI"
- **SI_CONTRARIO:** "CONTRARIO" (bloque anidado)
- **SI_SINO:** "SINO" (bloque alternativo si el "SI" es falso)
- **SEGUN_CONDICIONAL:** "SEGUN"
- **CASO_SEGUN:** "CASO"
- **DEFECTO_SEGUN:** "DEFECTO"
- **DETENER_SEGUN:** "DETENER"

Estructuras de Ciclo

- **PARA_CICLO:** "PARA"
- **MIENTRAS_CICLO:** "MIENTRAS"

Entrada y Salida

- **LEER_FUNCION:** "LEER"
- **IMPRIMIR_FUNCION:** "IMPRIMIR"

Operadores de Incremento y Decremento

- **INCREMENTO_AL_VALOR:** "++"
- **DECREMENTO_AL_VALOR:** "--"

Operadores Aritméticos

- **SUMA:** "+"
- **RESTA:** "-"
- **MULTIPLICACION:** "*"
- **DIVISION:** "/"

Operadores Adicionales

- **MODULO:** "%"
- **EXPONENTE:** "**"

Operadores de Comparación y Asignación

- **IGUAL_ASIGNACION:** "="
- **IGUAL_COMPARACION:** "=="
- **DIFERENTE:** "!="
- **MENOR:** "<"
- **MENOR_IGUAL:** "<="
- **MAYOR:** ">"
- **MAYOR_IGUAL:** ">="



Operadores Lógicos

- **Y_LOGICO:** "&&"
- **O_LOGICO:** "||"
- **NO_LOGICO:** "!!"

Delimitadores

- **ABRIR_LLAVE:** "{"
- **CERRAR_LLAVE:** "}"
- **ABRIR_PARENTESIS:** "("
- **CERRAR_PARENTESIS:** ")"
- **ABRIR_CORCHETE:** "["
- **CERRAR_CORCHETE:** "]"
- **COMA:** ","
- **DOS_PUNTOS:** ":"
- **PUNTO_COMA:** ";"

GRÁMATICAS

Sentencias

```
void Sentencias() :  
{  
  {  
    (  
      DeclaracionVariable()  
    | DeclaracionArreglo()  
    | DeclaracionMatriz()  
    | Asignacion()  
    | Imprimir()  
    | Leer()  
    | CondicionalSi()  
    | CondicionalSegun()  
    | BuclePara()  
    | BucleMientras()  
    | BloqueAnonimo()  
    /*  
     Bloque anónimo para atrapar llaves de estructuras de control que fallaron  
     y evitar que desbalancen las llaves de todo el programa.  
    */  
    | <PUNTO_COMA>  
    //Sentencia Vacía: Solo un punto y coma, para permitir sentencias vacías sin romper el programa.  
    )*  
  }  
}
```

Img 12 Gramática de sentencias

Explicación:

- Define todos los tipos de instrucciones ejecutables y válidas dentro del lenguaje.
- Incluye un BloqueAnonimo() como mecanismo de seguridad (recuperación de errores) para atrapar fallos en bloques de código y evitar el desbalanceo general de las llaves en el programa.

Bloque anónimo

```
// Bloque anónimo para recuperación de errores  
void BloqueAnonimo() :  
{  
  {  
    <ABRIR_LLAVE> Sentencias() <CERRAR_LLAVE>  
  }  
}
```

Img 13 Gramática para recuperación de errores

Explicación:

- Es una estructura auxiliar diseñada de forma exclusiva para la recuperación de errores sintácticos.
- Agrupa las sentencias encerradas entre llaves { } con el objetivo de atrapar llaves "huérfanas" dejadas por estructuras de control que hayan fallado en su sintaxis.
- Evita que un error local corrompa el balance de llaves de todo el código principal, permitiendo que el compilador siga analizando el resto del programa.

Programa principal

```
void Programa() :
{
{
    try{
        <INICIO_PROGRAMA>
        <ABRIR_LLAVE>

        Sentencias()

        <CERRAR_LLAVE>
        <FIN_PROGRAMA>
        <EOF>
    }catch(ParseException e){
        int[] pos = obtenerPosicion(e);

        String detalleDinamico = obtenerDetalleSintactico(e);

        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
            detalleDinamico,
            consejo: "Todo código debe comenzar con INICIO { y terminar con } FIN."));

        // Como es la regla raíz consumimos hasta EOF para no generar más ruido
        while (getNextToken().kind != EOF) {}
    }
}
}
```

Img 14 Gramática de programa principal

Explicación:

- Define el punto de entrada, el cuerpo y el punto de salida estructurado del código.
- Todo programa de *EasyScript* debe comenzar estrictamente con la palabra reservada INICIO, abrir llaves {, contener las sentencias y cerrar con } seguido de la palabra reservada FIN.

Tipo de dato

```
void TipoDato() :  
{  
  {  
    <ENTERO_DATO>  
    | <DECIMAL_DATO>  
    | <BOOLEANO_DATO>  
    | <CARACTER_DATO>  
    | <TEXTO_DATO>  
  }  
}
```

Img 15 Gramática para tipos de datos

Explicación:

- Agrupa las palabras reservadas que definen los tipos de variables soportados por el compilador.
- Reconoce los tipos fundamentales: ENT (Entero), DEC (Decimal), BOOL (Booleano), LETRA (Carácter) y TXT (Texto).



Valores

```
void Valor() :
{
    Token err;
}
{
    <NUM_ENTERO>
    | <NUM_DECIMAL>
    | <CADENA>
    | <VALOR_LETRA>

    | <VERDADERO_BOOLEANO>
    | <FALSO_BOOLEANO>
    | <VARIABLE>
    | <CONSTANTE>

    // Atrapa errores Léxicos

    | err = <ERROR_TEXTO>
    {
        // Acción Léxica: Esto se ejecuta al instante
        EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
            "Cadena de texto sin cerrar.",
            "Cierra la cadena de texto con comillas dobles."));
    }
    | err = <ERROR_CARACTER>
    {
        // Acción Léxica: Esto se ejecuta al instante
        EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
            "Caracter sin cerrar.",
            "Cierra el caracter con comillas simples."));
    }
}
```

Img 16 Gramática de valores



```
| err = <ERROR_NUMERO_INVALIDO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Formato decimal invalido '" + err.image + "'",
        "Sigue la regla para decimales. Ejemplo: 3.1416"));
}

| err = <ERROR_IDENTIFICADOR_INVALIDO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Identificador no valido '" + err.image + "'",
        "Toda variable/constante debe iniciar con una mayúscula seguido de minúsculas,
        no debe empezar con números. Ejemplo: 'Variable_1' o 'CONST_Mi_Nombre'"));
}

| err = <ERROR_LONGITUD_CARACTER>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Formato de caracter invalido: " + err.image,
        "Longitud mayor a 1 o caracter no reconocido para LETRA. Ejemplos: 'a' o '5'"));
}

| err = <ERROR_LOGICO_INCOMPLETO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Operador incompleto o no valido: '" + err.image + "'",
        "Los operadores en EasyScript son dobles (&&, ||, !!)."));
}
}
```

Img 17 Gramática de valores, posibles errores

Explicación:

- Reconoce cualquier tipo de valor válido que puede ser asignado u operado, incluyendo literales numéricos, cadenas de texto, caracteres, valores booleanos (VERDADERO, FALSO), así como identificadores (variables o constantes).



Expresión aritmética

```
void ExpresionAritmetica() :  
{  
{  
    ExpresionNivel1()  
}  
  
// Nivel 1: Maneja la suma y la resta  
void ExpresionNivel1() :  
{  
{  
    ExpresionNivel2()  
    (  
        ( <SUMA> | <RESTA> )  
        ExpresionNivel2()  
    )*  
}  
  
// Nivel 2: Maneja multiplicación, división y módulo  
void ExpresionNivel2() :  
{  
{  
    ExpresionNivel3()  
    (  
        ( <MULTIPLICACION> | <DIVISION> | <MODULO> )  
        ExpresionNivel3()  
    )*  
}  
}
```

Img 18 Gramática de expresión aritmética

```
// Level 3: Maneja exponentes
void ExpresionNivel3() :
{
{
    ExpresionNivel4()
    (
        <EXPONENTE>
        ExpresionNivel4()
    ) *
}
}

// Level 4: Maneja operadores unarios (números negativos) y valores/paréntesis
void ExpresionNivel4() :
{
{
    ( <RESTA> )? // Soporte para número negativo unario
    ValorConArreglos()
}
}

// Subnivel para manejar valores, llamadas a arreglos o expresiones entre paréntesis
void ValorConArreglos() :
{
{
    (
        <ABRIR_PARENTESIS> ExpresionAritmetica() <CERRAR_PARENTESIS>
    |
        Valor()
        // Soporte opcional para acceder a índices de arreglos o matrices
        (
            <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE>
            ( <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE> )?
        )?
    )
}
}
```

Img 19 Gramática de expresión aritmética

Explicación:

- Define la estructura jerárquica para resolver operaciones matemáticas, respetando de manera estricta la precedencia matemática de operadores mediante subniveles (Nivel 1 al Nivel 4).
- Maneja desde sumas y restas (menor precedencia), multiplicaciones, divisiones y módulos, hasta el uso de exponentes y operadores unarios (números negativos).
- Permite el uso de paréntesis para alterar la precedencia natural y da soporte al acceso de valores dentro de arreglos y matrices mediante corchetes [].



Declaracion de variable

```
void DeclaracionVariable() :
{
    Token id; // Variable temporal para guardar el token
}
{
    try {
        TipoDato()

        ( id = <VARIABLE> | id = <CONSTANTE> | id = <ERROR_IDENTIFICADOR_INVALIDO> )

        {
            // Validamos y guardamos el error léxico sin detener la compilación
            if (id.kind == ERROR_IDENTIFICADOR_INVALIDO) {
                EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", id.beginLine, id.beginColumn,
                    "Identificador no valido '" + id.image + "'",
                    "Toda variable/constante debe iniciar con una mayúscula seguido de minúsculas,
                    no debe empezar con números. Ejemplo: 'Variable_1' o 'CONST_Mi_Nombre'"));
            }
        }

        ( <IGUAL_ASIGNACION> ExpresionAritmetica() )?
        <PUNTO_COMA>
    }
}
```

Img 20 Gramática para declaración de variable

```
} catch (ParseException e) {
    int[] pos = obtenerPosicion(e);

    String detalleDinamico = obtenerDetalleSintactico(e);

    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
        detalleDinamico,
        consejo: "Asegúrate de definir el tipo de dato, el nombre correctamente y terminar con un punto y coma."));

    // Usamos Sync Tokens para detenernos si vemos el inicio de otra sentencia
    recuperarHastaSync(SYNC_SENTENCIAS);
}
}
```

Img 21 Gramática para declaración de variable, posible error

Explicación:

- Define la sintaxis para crear nuevas variables o constantes. Debe iniciar con un tipo de dato válido, seguido del identificador.
- Permite de manera opcional realizar una asignación de valor inicial inmediata, y requiere finalizar forzosamente con un punto y coma ;.



Declaración de arreglo

```
void DeclaracionArreglo() :
{ Token id; }
{
    try {
        <ARREGLO_ESTRUCTURA> TipoDato()

        ( id = <VARIABLE> | id = <ERROR_IDENTIFICADOR_INVALIDO> )
        {
            if (id.kind == ERROR_IDENTIFICADOR_INVALIDO) {
                EasyCompiler.listaErrores.add(new ErrorCompilador(tipo: "Léxico", id.beginLine, id.beginColumn,
                    "Identificador no valido '" + id.image + "'",
                    consejo: "Toda variable debe iniciar con mayúscula."));
            }
        }

        // El tamaño del arreglo va entre corchetes
        <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE>

        //Iniciación del Arreglo Opcional
        (
            <IGUAL_ASIGNACION> <ABRIR_LLAVE> ExpresionAritmetica() (<COMA> ExpresionAritmetica())* <CERRAR_LLAVE> )?

            <PUNTO_COMA>
        } catch (ParseException e) {
            int[] pos = obtenerPosicion(e);
            String detalleDinamico = obtenerDetalleSintactico(e);
            listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
                detalleDinamico,
                "Estructura inválida. Usa: ARREGLO Tipo Nombre[Tamaño];"));
            recuperarHastaSync(SYNC_SENTENCIAS);
        }
    }
}
```

Img 22 Gramática para declaración de arreglo

Explicación:

- Establece la sintaxis para crear arreglos unidimensionales. Utiliza la palabra reservada ARREGLO, un tipo de dato, el nombre del arreglo y define su tamaño entre corchetes [].
- Admite opcionalmente la inicialización directa de los valores del arreglo encerrándolos entre llaves { } y separados por comas.



Declaración de matriz

```
void DeclaracionMatriz() :
{ Token id; }
{
    try {
        <MATRIZ_ESTRUCTURA> TipoDato()

        ( id = <VARIABLE> | id = <ERROR_IDENTIFICADOR_INVALIDO> )
        {
            if (id.kind == ERROR_IDENTIFICADOR_INVALIDO) {
                EasyCompiler.listaErrores.add(new ErrorCompilador(tipo: "Léxico", id.beginLine, id.beginColumn,
                    "Identificador no valido '" + id.image + "'",
                    consejo: "Toda variable debe iniciar con mayúscula."));
            }
        }

        // Las matrices requieren dos dimensiones
        <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE>
        <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE>

        (
            <IGUAL_ASIGNACION> <ABRIR_LLAVE>
            <ABRIR_LLAVE> ExpresionAritmetica() (<COMA> ExpresionAritmetica())* <CERRAR_LLAVE>
            ( <COMA> <ABRIR_LLAVE> ExpresionAritmetica() (<COMA> ExpresionAritmetica())* <CERRAR_LLAVE> )*
            <CERRAR_LLAVE>
        )?
        <PUNTO_COMA>
    } catch (ParseException e) {
        int[] pos = obtenerPosicion(e);
        String detalleDinamico = obtenerDetalleSintactico(e);
        listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
            detalleDinamico,
            "Estructura inválida. Usa: MATRIZ Tipo Nombre[Filas][Columnas];"));
        recuperarHastaSync(SYNC_SENTENCIAS);
    }
}
```

Img 23 Gramática para declaración de matriz

Explicación:

- Define la estructura para crear colecciones bidimensionales. Usa la palabra reservada MATRIZ, seguida del tipo de dato, el identificador y sus dos dimensiones especificadas mediante corchetes adyacentes [][].
- Permite la inicialización explícita utilizando llaves anidadas para filas y columnas.
- Al igual que otras declaraciones, implementa captura de excepciones sintácticas para evitar la caída del analizador en caso de una mala estructura.



Asignación

```
void Asignacion() :
{
    Token id;
}
{
    try {
        ( id = <VARIABLE> | id = <CONSTANTE> | id = <ERROR_IDENTIFICADOR_INVALIDO> )
        {
            // Validamos qué fue lo que entró
            if (id.kind == ERROR_IDENTIFICADOR_INVALIDO) {
                EasyCompiler.listaErrores.add(new ErrorCompilador(tipo: "Léxico", id.beginLine, id.beginColumn,
                    "Asignación incorrecta: Identificador no valido '" + id.image + "'",
                    consejo: "Toda variable debe iniciar con mayúscula y no contener caracteres especiales."));
            }
        }
        //Soporte opcional para acceder a índices de arreglos o matrices
        (
            <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE>
            ( <ABRIR_CORCHETE> ExpresionAritmetica() <CERRAR_CORCHETE> )? // El segundo es opcional (para matrices)
        )?

        // Puede ser una asignación (= 5) o un incremento/decremento (++)
        (
            ( <IGUAL_ASIGNACION> ExpresionAritmetica() )
            | <INCREMENTO_AL_VALOR>
            | <DECREMENTO_AL_VALOR>
        )
        <PUNTO_COMA>
    } catch (ParseException e) {
        int[] pos = obtenerPosicion(e);

        String detalleDinamico = obtenerDetalleSintactico(e);
        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
            detalleDinamico,
            consejo: "Asegúrate de asignar un valor válido y terminar con un punto y coma."));
        recuperarHastaSync(SYNC_SENTENCIAS);
    }
}
```

Img 24 Gramática para asignación

Explicación:

- Permite actualizar o reasignar el valor de una variable o constante ya existente, e incluye soporte para actualizar el valor en un índice específico de un arreglo o matriz (usando corchetes []).
- Reconoce tanto la asignación directa (usando el operador =) como los operadores abreviados de incremento (++) y decremento (--).
- Implementa la validación del identificador y un bloque de recuperación ante fallos de sintaxis en el valor a asignar.

Regla para el contenido dentro de IMPRIMIR ()

```
void ExpresionImprimir() :  
{  
{  
    Valor() ( <SUMA> Valor() ) *  
}  
}
```

Img 25 Regla para IMPRIMIR

Explicación:

- Define la concatenación de múltiples componentes de salida.
- Permite unir uno o más valores (cadenas de texto, números enteros, decimales, variables, etc.) utilizando únicamente el operador de suma (+) como mecanismo de concatenación para la terminal.

Imprimir

```
void Imprimir() :  
{  
{  
    try {  
        <IMPRIMIR_FUNCION> <ABRIR_PARENTESIS> ExpresionImprimir() <CERRAR_PARENTESIS> <PUNTO_COMA>  
    } catch (ParseException e) {  
        int[] pos = obtenerPosicion(e);  
        String detalleDinamico = obtenerDetallesSintactico(e);  
        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
            detalleDinamico,  
            consejo: "Asegúrate de usar la estructura IMPRIMIR(valor);"));  
        recuperarHastaSync(SYNC_SENTENCIAS);  
    }  
}
```

Img 26 Gramática para imprimir

Explicación:

- Analiza una instrucción de salida de datos a consola.
- Comprueba estrictamente la palabra clave IMPRIMIR seguida de paréntesis envolviendo la expresión (texto o variable concatenada) y terminando obligatoriamente con punto y coma ;.
- Su bloque try-catch asegura que, ante un paréntesis olvidado o mala concatenación, el analizador pueda continuar con el resto de las líneas.

Leer

```
void Leer() :
{
{
    try {
        <LEER_FUNCION> <ABRIR_PARENTESIS> <VARIABLE> <CERRAR_PARENTESIS> <PUNTO_COMA>
    } catch (ParseException e) {
        int[] pos = obtenerPosicion(e);
        String detalleDinamico = obtenerDetalleSintactico(e);
        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
            detalleDinamico,
            consejo: "La estructura correcta es LEER(Variable);"));
        recuperarHastaSync(SYNC_SENTENCIAS);
    }
}
```

Img 27 Gramatica para leer

Explicación:

- Analiza una instrucción diseñada para solicitar entrada de datos por teclado.
- Utiliza la palabra reservada LEER seguida estrictamente de un identificador de variable dentro de paréntesis y un punto y coma final (no acepta constantes).
- Protege la compilación en caso de sintaxis errónea informándole al usuario la estructura correcta esperada LEER(Variable);.

Operador relacional

```
void OperadorRelacional() :
{
{
    <MAYOR> | <MENOR> | <IGUAL_COMPARACION> | <DIFERENTE> | <MAYOR_IGUAL> | <MENOR_IGUAL>
}
}
```

Img 28 Gramática para operador relacional

Explicación:

- Agrupa y reconoce cualquiera de los operadores utilizados para comparar valores matemáticos o lógicos.
- Incluye los símbolos de: mayor >, menor <, igual de comparación ==, diferente !=, mayor o igual >=, menor o igual <=.



Condición simple

```
void CondicionSimple() :
{ Token err = null; }
{
    // 1. Opcional: Puede iniciar con la negación lógica (!!) o el error (!)
    (
        <NO_LOGICO>
        | err = <ERROR_NEGACION_INCOMPLETA>
        {
            EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
                "Operador de negación incompleto: '" + err.image + "'",
                "Para negar una condición en EasyScript debes usar '!!'.");
        }
    )?

    // 2. La expresión principal a evaluar
    ExpresionAritmetica()

    // 3. Opcional: Operador relacional y segundo valor (Ej. > 5)
    ( OperadorRelacional() Valor() )?
}
}
```

Img 29 Gramática para condición simple

Explicación:

- Es la unidad más básica de una evaluación lógica.
- Evalúa un valor principal, y opcionalmente puede compararlo con un segundo valor usando algún operador relacional.
- Cuenta con soporte para invertir lógicamente la evaluación mediante la lectura del operador unario de negación (!!). Captura también los intentos inválidos de negación.

Condición compuesta

```
void Condicion() :
{ Token err = null; }
{
    // Siempre debe haber al menos una condición simple
    CondicionSimple()

    // Cero o más veces, puede estar unida a otra por medio de AND / OR
    (
        // Leemos el operador (&&, ||) o el error de escribir solo (&, |)
        (
            <Y_LOGICO>
            | <O_LOGICO>
            | err = <ERROR_LOGICO_INCOMPLETO>
            {
                // Atrapamos el '&' o '|' solitario en tiempo real
                EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
                    "Operador lógico incompleto: '" + err.image + "'",
                    "Asegúrate de usar operadores dobles: && para 'Y', || para 'O'."));
            }
        )
        // Y exigimos que después del operador venga otra condición
        CondicionSimple()
    )*
}
}
```

Img 30 Gramática para condición compuesta

Explicación:

- Permite enlazar múltiples "condiciones simples" para formar evaluaciones lógicas mucho más complejas.
- Se estructuran a través de los operadores de unión "Y" (&&) y de alternancia "O" (||).

Condicional SI | CONTRARIO | SINO

```
void CondicionalSi() :
{
{
try{
<SI_CONDICIONAL> <ABRIR_PARENTESIS> Condicion() <CERRAR_PARENTESIS> <ABRIR_LLAVE>
    Sentencias()
    <CERRAR_LLAVE>

    // EL CONTRARIO (Else If) es opcional y puede repetirse (*)
    (
        <SI_CONTRARIO> <ABRIR_PARENTESIS> Condicion() <CERRAR_PARENTESIS> <ABRIR_LLAVE>
            Sentencias()
            <CERRAR_LLAVE>
        )*

    // EL SINO (Else) es opcional y solo aparece una vez (?)
    (
        <SI_SINO> <ABRIR_LLAVE>
            Sentencias()
            <CERRAR_LLAVE>
        )?
    }catch(ParseException e){
        int[] pos = obtenerPosicion(e);
        String detalleDinamico = obtenerDetalleSintactico(e);

        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
            detalleDinamico,
            consejo: "Verifica que la condición lógica esté completa y encierra las sentencias entre { }.");
        // consumirPuntoYComa = false → nos detenemos en "}" sin consumirlo
        // para no romper el bloque padre.
        recuperarHastaSync(SYNC_SENTENCIAS);
        if (getToken(1).kind == CERRAR_LLAVE) {
            getNextToken();
        }
    }
}
}
```

Img 31 Gramática para condicional SI | CONTRARIO | SINO

Explicación:

- Analiza la principal estructura condicional del lenguaje. Se encarga de evaluar una condición central; si resulta ser verdadera, ejecuta el bloque de sentencias delimitadas por llaves { }.
- Habilita el uso opcional e ilimitado de bloques intermedios CONTRARIO (para evaluar sub-condiciones) y admite la finalización con un bloque único SINO que se ejecuta si todo lo anterior resulta falso.

- Protege la ejecución de todo el sistema asegurando el cierre ordenado de las llaves, incluso cuando hay errores de sintaxis en medio.

Condicional SEGÚN

```
void CondicionalSegun() :  
{  
{  
    try{  
        <SEGUN_CONDICIONAL> <ABRIR_PARENTESIS> <VARIABLE> <CERRAR_PARENTESIS> <ABRIR_LLAVE>  
  
        // Debe haber al menos un CASO (+)  
        (  
            <CASO_SEGUN> Valor() <DOS_PUNTOS>  
            Sentencias()  
            <DETENER_SEGUN> <PUNTO_COMA>  
        )+  
  
        // EL DEFECTO es opcional (?)  
        (  
            <DEFECTO_SEGUN> <DOS_PUNTOS>  
            Sentencias()  
        )?  
  
        <CERRAR_LLAVE>  
  
    }catch(ParseException e){  
        int[] pos = obtenerPosicion(e);  
        String detalleDinamico = obtenerDetalleSintactico(e);  
        listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
            detalleDinamico,  
            consejo: "El SEGUN requiere 'CASO valor:', finalizar cada caso con 'DETENER;' y cerrar llaves."));  
        recuperarHastaSync(SYNC_SENTENCIAS);  
        if (getToken(1).kind == CERRAR_LLAVE) {  
            getNextToken();  
        }  
    }  
}  
}
```

Img 32 Gramática para condicional SEGÚN

Explicación:

- Estructura de selección múltiple enfocada en analizar múltiples estados de una variable definida entre paréntesis.
- Exige por regla incluir al menos un bloque CASO el cual debe finalizar su ejecución obligatoriamente invocando la palabra DETENER;.
- Permite definir una ejecución alternativa de cierre empleando la palabra DEFECTO:.
- Informa al usuario exactamente cómo estructurar sus casos cuando el analizador detecta una omisión en los delimitadores requeridos.

Bucle PARA

```
void BuclePara() :
{
{
    try{
        <PARA_CICLO> <ABRIR_PARENTESIS>
        // 1. Declaración inicial (Ej. ENT i = 0;)
        DeclaracionVariable()

        // 2. Condición (Ej. i < 10)
        Condicion() <PUNTO_COMA>

        // 3. Incremento/Decremento (Asumiendo tokens como INCREMENTO para "++")
        <VARIABLE> ( <INCREMENTO_AL_VALOR> | <DECREMENTO_AL_VALOR> )

        <CERRAR_PARENTESIS> <ABRIR_LLAVE>
        Sentencias()
        <CERRAR_LLAVE>
    }catch(ParseException e){
        int[] pos = obtenerPosicion(e);
        String detalleDinamico = obtenerDetalleSintactico(e);
        listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
            detalleDinamico,
            "El ciclo necesita 3 partes: Declaración ; Condición ; Incremento/Decremento."));
        recuperarHastaSync(SYNC_SENTENCIAS);
        if (getToken(1).kind == CERRAR_LLAVE) {
            getNextToken();
        }
    }
}
}
```

Img 33 Gramática para bucle PARA

Explicación:

- Analiza un ciclo de repetición determinado el cual requiere tres componentes obligatorios separados por punto y coma dentro de los paréntesis:
 1. Declaración inicial.
 2. Condición de evaluación.
 3. Instrucción de incremento o decremento (ej. I++).



Bucle MIENTRAS

```
void BucleMientras() :
{
{
    try {
        <MIENTRAS_CICLO> <ABRIR_PARENTESIS> Condicion() <CERRAR_PARENTESIS> <ABRIR_LLAVE>
            Sentencias()
        <CERRAR_LLAVE>
    } catch (ParseException e) {
        int[] pos = obtenerPosicion(e);
        String detalleDinamico = obtenerDetalleSintactico(e);
        listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
            detalleDinamico,
            "El ciclo necesita la estructura: MIENTRAS ( Condicion ) { sentencias }."));

        // Falso para detenernos en la llave de cierre sin consumirla (protege el bloque padre)
        recuperarHastaSync(SYNC_SENTENCIAS);
        if (getToken(1).kind == CERRAR_LLAVE) {
            getNextToken();
        }
    }
}
}
```

Img 34 Gramática para bucle MIENTRAS

Explicación:

- Estructura un ciclo de repetición indeterminada que evalúa la veracidad de la condición declarada entre paréntesis.
- Si la condición es cierta, ejecuta las sentencias atrapadas entre las llaves; de lo contrario termina el ciclo.
- Aporta un manejo de excepciones que reporta al usuario cuando olvida la condición o el cierre y apertura de los bloques de este ciclo en particular.

MANEJO DE ERRORES

Creación de lista para almacenar errores

```
PARSER_BEGIN(EasyCompiler)

import java.util.ArrayList; // Agregado para usar listas dinámicas
import java.util.List;      // Agregado para usar listas dinámicas

public class EasyCompiler {
    // Declaramos la lista de forma estática para que la clase Main pueda imprimirla al final
    public static List<ErrorCompilador> listaErrores = new ArrayList<>();

    // Tokens de sincronización para inicio de sentencias
    static final int[] SYNC_SENTENCIAS = {
        ENTERO_DATO, DECIMAL_DATO, BOOLEANO_DATO, CARACTER_DATO, TEXTO_DATO,
        ARREGLO_ESTRUCTURA, MATRIZ_ESTRUCTURA, IMPRIMIR_FUNCION, LEER_FUNCION,
        SI_CONDICIONAL, SEGUN_CONDICIONAL, PARA_CICLO, MIENTRAS_CICLO,
        ABRIR_LLAVE // Token para evitar que llaves huérfanas rompan el programa
    };
}
```

Img 35 Lista para manejo de errores

Explicación:

- Consiste en la implementación de una estructura de datos global (como un `ArrayList<String>`) encargada de recolectar todos los incidentes detectados durante las fases de análisis léxico y sintáctico.
- En lugar de detener la ejecución al primer fallo, el compilador almacena el mensaje de error y continúa el proceso. Esto permite generar un reporte final consolidado que muestra al programador todos los puntos a corregir de una sola vez, optimizando el tiempo de depuración.

Método para obtener posición de error

```
private static int[] obtenerPosicion(ParseException e) {
    Token t = (e.currentToken != null && e.currentToken.next != null)
        ? e.currentToken.next
        : e.currentToken;
    // Si por alguna razón ambos son null devolvemos -1 para evitar crash
    if (t == null) return new int[]{-1, -1};
    return new int[]{t.beginLine, t.beginColumn};
}
```

Img 36 Método para ubicar error

Explicación:

- Es la función encargada de extraer las coordenadas exactas (línea y columna) donde el analizador detectó una anomalía.
- Utiliza los atributos internos del objeto Token (como beginLine y beginColumn). Esta precisión es fundamental para la usabilidad del sistema, ya que guía al usuario directamente al origen del problema en el código fuente, evitando búsquedas manuales en archivos extensos.

Recuperación de pánico con Sync Tokens

```
private void recuperarHastaSync(int... syncTokens) {
    Token t = getToken(1);
    while (t.kind != EOF) {
        boolean encontrado = false;
        for (int st : syncTokens) {
            if (t.kind == st) {
                encontrado = true;
                break;
            }
        }
        if (encontrado || t.kind == CERRAR_LLAVE || t.kind == PUNTO_COMA) {
            break;
        }
        getNextToken();
        t = getToken(1);
    }

    // Si nos detuvimos en un punto y coma, lo consumimos para limpiar la sentencia.
    if (getToken(1).kind == PUNTO_COMA) {
        getNextToken();
    }
}
```

Img 37 Método para recuperación de pánico

Explicación:

- Es una técnica de resiliencia que se activa tras capturar una excepción sintáctica. Cuando el compilador encuentra un error en una sentencia, entra en un estado de "búsqueda" donde ignora todos los tokens entrantes hasta encontrar un símbolo de sincronización predefinido, generalmente el punto y coma (;).
- Una vez localizado este símbolo, el analizador recupera el control y reinicia la validación gramatical en la siguiente línea, evitando que un error aislado genere una cascada de fallos falsos en el resto del documento.

Extracción del token que fallo y cual se esperaba

```
private static String obtenerDetalleSintactico(ParseException e) {
    if (e.currentToken == null || e.currentToken.next == null) return "Error de sintaxis desconocido.";

    // 1. Obtenemos lo que el compilador leyó por error
    String tokenEncontrado = e.currentToken.next.image;
    if (e.currentToken.next.kind == EOF) {
        tokenEncontrado = "Fin del archivo";
    }

    // 2. Extraemos lo que JavaCC estaba esperando
    StringBuilder tokensEsperados = new StringBuilder();
    int maxSize = 0;

    // JavaCC guarda las expectativas en un arreglo 2D
    for (int i = 0; i < e.expectedTokenSequences.length; i++) {
        if (maxSize < e.expectedTokenSequences[i].length) {
            maxSize = e.expectedTokenSequences[i].length;
        }
        for (int j = 0; j < e.expectedTokenSequences[i].length; j++) {
            // tokenImage traduce el ID del token a su texto real (ej. ";" o "<VARIABLE>")
            tokensEsperados.append(e.tokenImage[e.expectedTokenSequences[i][j]]).append(str: " ");
        }
        if (i < e.expectedTokenSequences.length - 1) {
            tokensEsperados.append(str: " o ");
        }
    }

    // Limpiamos las comillas extrañas que pone JavaCC por defecto
    String esperadosStr = tokensEsperados.toString().replace(target: "\"", replacement: "");

    return "Se encontró '" + tokenEncontrado + "', pero se esperaba: '" + esperadosStr;
}

private void agregarError(String tipo, Token token, String mensaje, String sugerencia) {
    if (token != null) {
        EasyCompiler.listaErrores.add(new ErrorCompilador(tipo, token.beginLine, token.beginColumn, mensaje, sugerencia));
    } else {
        EasyCompiler.listaErrores.add(new ErrorCompilador(tipo, -1, -1, mensaje, sugerencia));
    }
}
```

Img 38 Extracción de token erróneo

Explicación:

- Este proceso se encarga de contrastar el token que el analizador recibió frente al token que la gramática esperaba encontrar en esa posición específica.
- Al ocurrir la discrepancia, el sistema extrae la imagen del token actual (lo que el usuario escribió) y genera un mensaje informativo que explica la contradicción. Por ejemplo, si el sistema espera un operador pero recibe un identificador, el mensaje señalará claramente esta confusión para orientar la corrección.

Errores Léxicos

Se presentan a nivel de caracteres individuales. Ocurren cuando el TokenManager encuentra un símbolo que no está definido en el alfabeto del lenguaje (como caracteres especiales no permitidos) o cuando una secuencia no logra formar un token válido (como una cadena de texto que no se cierra con comillas).

Estos errores impiden la creación de los componentes básicos necesarios para el análisis posterior.

Error de texto

```
err = <ERROR_TEXTO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Cadena de texto sin cerrar.",
        "Cierra la cadena de texto con comillas dobles."));
}
```

Img 39 Error de texto

Error de caracter

```
err = <ERROR_CARACTER>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Caracter sin cerrar.",
        "Cierra el caracter con comillas simples."));
}
```

Img 40 Error de caracter

Error de número invalido

```
err = <ERROR_NUMERO_INVALIDO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Formato decimal invalido '" + err.image + "'",
        "Sigue la regla para decimales. Ejemplo: 3.1416"));
}
```

Img 41 Error de número invalido



Error de identificador invalido

```
err = <ERROR_IDENTIFICADOR_INVALIDO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Identificador no valido '" + err.image + "'",
        "Toda variable/constante debe iniciar con una mayúscula seguido de minúsculas,
        no debe empezar con números. Ejemplo: 'Variable_1' o 'CONST_Mi_Nombre'"));
}
```

Img 42 Error de identificador invalido

Error de longitud en caracter

```
err = <ERROR_LONGITUD_CARACTER>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Formato de caracter invalido: " + err.image,
        "Longitud mayor a 1 o caracter no reconocido para LETRA. Ejemplos: 'a' o '5'"));
}
```

Img 43 Error de longitud en caracter

Error de comparador lógico incompleto

```
err = <ERROR_LOGICO_INCOMPLETO>
{
    // Acción Léxica: Esto se ejecuta al instante
    EasyCompiler.listaErrores.add(new ErrorCompilador("Léxico", err.beginLine, err.beginColumn,
        "Operador incompleto o no valido: '" + err.image + "'",
        "Los operadores en EasyScript son dobles (&&, ||, !!)."));
}
```

Img 44 Error de comparador lógico incompleto



Errores Sintácticos

Surgen cuando, a pesar de que los tokens son válidos individualmente, su combinación no respeta las reglas de estructura del lenguaje.

Ejemplos comunes incluyen la omisión de paréntesis en una condición, el orden incorrecto en una declaración de variable o la falta de palabras reservadas obligatorias. Estos errores son capturados por las producciones gramaticales mediante bloques try-catch para activar los mecanismos de recuperación.

Error flujo de programa

```
}catch(ParseException e){  
    int[] pos = obtenerPosicion(e);  
  
    String detalleDinamico = obtenerDetalleSintactico(e);  
  
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        consejo: "Todo código debe comenzar con INICIO { y terminar con } FIN."));  
  
    // Como es la regla raíz consumimos hasta EOF para no generar más ruido  
    while (getNextToken().kind != EOF) {}  
}
```

Img 45 Error de flujo de programa

Erro declaración de variable

```
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
  
    String detalleDinamico = obtenerDetalleSintactico(e);  
  
    listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        "Asegúrate de definir el tipo de dato, el nombre correctamente y terminar con un punto y coma."));  
  
    // Usamos Sync Tokens para detenernos si vemos el inicio de otra sentencia  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 46 Error de declaración de variables



Error declaración de arreglo

```
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
    String detalleDinamico = obtenerDetalleSintactico(e);  
    listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        "Estructura inválida. Usa: ARREGLO Tipo Nombre[Tamaño];"));  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 47 Error de declaración de arreglo

Error declaración de matriz

```
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
    String detalleDinamico = obtenerDetalleSintactico(e);  
    listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        "Estructura inválida. Usa: MATRIZ Tipo Nombre[Filas][Columnas];"));  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 48 Error de declaración de matriz

Error de asignación

```
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
  
    String detalleDinamico = obtenerDetalleSintactico(e);  
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        consejo: "Asegúrate de asignar un valor válido y terminar con un punto y coma."));  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 49 Error de asignación



Error de impresión

```
try {  
    <IMPRIMIR_FUNCION> <ABRIR_PARENTESIS> ExpresionImprimir() <CERRAR_PARENTESIS> <PUNTO_COMA>  
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
    String detalleDinamico = obtenerDetalleSintactico(e);  
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        consejo: "Asegúrate de usar la estructura IMPRIMIR(valor);"));  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 50 Error de impresión

Error de lectura

```
} catch (ParseException e) {  
    int[] pos = obtenerPosicion(e);  
    String detalleDinamico = obtenerDetalleSintactico(e);  
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        consejo: "La estructura correcta es LEER(Variable);"));  
    recuperarHastaSync(SYNC_SENTENCIAS);  
}
```

Img 51 Error de lectura

Error condicional SI

```
}catch(ParseException e){  
    int[] pos = obtenerPosicion(e);  
    String detalleDinamico = obtenerDetalleSintactico(e);  
  
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],  
        detalleDinamico,  
        consejo: "Verifica que la condición lógica esté completa y encierra las sentencias entre { }."));  
    // consumirPuntoYComa = false → nos detenemos en "}" sin consumirlo  
    // para no romper el bloque padre.  
    recuperarHastaSync(SYNC_SENTENCIAS);  
    if (getToken(1).kind == CERRAR_LLAVE) {  
        getNextToken();  
    }  
}
```

Img 52 Error de condicional SI



Error condicional SEGUN

```
}catch(ParseException e){
    int[] pos = obtenerPosicion(e);
    String detalleDinamico = obtenerDetalleSintactico(e);
    listaErrores.add(new ErrorCompilador(tipo: "Sintáctico", pos[0], pos[1],
        detalleDinamico,
        consejo: "El SEGUN requiere 'CASO valor:', finalizar cada caso con 'DETENER;' y cerrar llaves."));
    recuperarHastaSync(SYNC_SENTENCIAS);
    if (getToken(1).kind == CERRAR_LLAVE) {
        getNextToken();
    }
}
```

Img 53 Error de condicional SEGUN

Error bucle PARA

```
}catch(ParseException e){
    int[] pos = obtenerPosicion(e);
    String detalleDinamico = obtenerDetalleSintactico(e);
    listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
        detalleDinamico,
        "El ciclo necesita 3 partes: Declaración ; Condición ; Incremento/Decremento."));
    recuperarHastaSync(SYNC_SENTENCIAS);
    if (getToken(1).kind == CERRAR_LLAVE) {
        getNextToken();
    }
}
```

Img 54 Error de bucle PARA

Error bucle MIENTRAS

```
} catch (ParseException e) {
    int[] pos = obtenerPosicion(e);
    String detalleDinamico = obtenerDetalleSintactico(e);
    listaErrores.add(new ErrorCompilador("Sintáctico", pos[0], pos[1],
        detalleDinamico,
        "El ciclo necesita la estructura: MIENTRAS ( Condicion ) { sentencias }.");

    // Falso para detenernos en la llave de cierre sin consumirla (protege el bloque padre)
    recuperarHastaSync(SYNC_SENTENCIAS);
    if (getToken(1).kind == CERRAR_LLAVE) {
        getNextToken();
    }
}
```

Img 55 Error de bucle MIENTRAS

CONCLUSIÓN

El desarrollo del compilador documentado en este manual representa mucho más que la construcción de un proyecto de software; es la culminación práctica de los fundamentos teóricos explorados a lo largo del curso. Durante la definición del lenguaje y la programación de la estructura en JavaCC, la teoría abstracta cobró un sentido completamente tangible.

Conceptos fundamentales como la implementación de Autómatas Finitos para la evaluación de tokens, el diseño meticuloso de Gramáticas Libres de Contexto (GLC) y la resolución de ambigüedades mediante jerarquías de operaciones, pasaron de ser modelos matemáticos en papel a convertirse en las reglas exactas que le dan vida y lógica a este analizador. Fases que inicialmente se estudiaron de manera independiente, como el análisis léxico y el análisis sintáctico, se integraron aquí para formar el motor que procesa y comprende cada línea de código.

Particularmente, el diseño del apartado de Manejo de Errores permitió llevar la teoría un paso más allá. La aplicación de técnicas como la recuperación en "Modo Pánico" y el uso de tokens de sincronización reforzaron una lección vital: el desarrollo de herramientas de sistema no se trata únicamente de procesar instrucciones perfectas, sino de diseñar software resiliente capaz de orientar al programador frente a sus propios fallos.

En definitiva, la creación de este compilador ha sido una experiencia integral que consolidó profundamente los temas vistos en clase. Ha demostrado de manera clara y aplicada que la materia de Lenguajes y Autómatas no es solo un requisito académico, sino el pilar estructural sobre el cual se sostiene toda la ingeniería de software y la creación de las tecnologías modernas.

REFERENCIAS BIBLIOGRÁFICAS

1. Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas* (2a ed.). Pearson Educación.
2. Appel, A. W. (2002). *Modern compiler implementation in Java* (2a ed.). Cambridge University Press.
3. Brookshear, J. G. (1993). *Teoría de la computación: Lenguajes formales, autómatas y complejidad*. Addison-Wesley Iberoamericana.
4. Cooper, K. D., & Torczon, L. (2011). *Engineering a compiler* (2a ed.). Morgan Kaufmann.
5. Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2007). *Introducción a la teoría de autómatas, lenguajes y computación* (3a ed.). Pearson Educación.
6. Martin, J. C. (2004). *Lenguajes formales y teoría de la computación* (3a ed.). McGraw-Hill.